

CS253 Assignment 1: Report

Name: Inesh Aggarwal

Roll Number: 240465

1. Design and Approach

The goal of this program is to process very large text files without using too much memory. If the program tried to load an entire massive file at once, it would be very slow and memory inefficient.

To solve this, the program reads the file incrementally using a fixed-size memory buffer. It loads a specific chunk of data, extracts the words, updates their counts, and then throws that raw chunk away before loading the next one. A "word" is treated as any continuous sequence of letters and numbers. To handle words that get chopped in half at the end of a chunk, the program stores the broken piece in a "left" string and attaches it to the front of the next chunk.

Everything is converted to lowercase to ensure matching is case-insensitive. Because the program only stores unique words and their counts in a dictionary format, memory usage stays low and only grows based on the number of unique words, which is independent of the actual file size.

2. Features and Functionalities

- **Word Count Query:** Returns the frequency of a word within a specific file version.
- **Top-K Query:** Displays the top 'K' most frequently occurring words in a specific version. The results are sorted in descending order of frequency, with alphabetical sorting used as a tie-breaker.
- **Difference Query:** Calculates the difference in the frequency of a word across two file versions.

3. Description of Classes Used

The system is built using distinct classes, each with a specific responsibility.

- **version_index:** This acts as the storage system for a specific version identifier. It holds the version's name and uses an unordered_map to link every unique word to the number of times it appears. It maintains a separate index for each version.
- **tokenizer:** This class processes the raw chunks of characters to extract valid words. It uses two overloaded functions. The main one directly updates the word counts in the system's memory by using a reference (freq[cur]++). It also uses a C++ feature called std::move to transfer strings efficiently without making slow copies in memory.
- **buffered_file_reader:** This class is responsible for safely opening the input file and reading it chunk-by-chunk. It strictly enforces the rule that the buffer must be between 256 KB and 1024 KB.

- **query hierarchy:** query is an abstract base class. From it, word_count_query, top_k_query, and difference_query are derived. Each specific class knows how to calculate the answer for its query type.
- **query_processor:** This class evaluates the queries. It demonstrates runtime polymorphism by executing the correct query logic dynamically at runtime.

4. Assumptions and Observations

- **Assumption on Top-K Sorting:** The requirements include displaying the top-K most frequent words in descending order. I assumed that if multiple words share the exact same frequency, they should be sorted alphabetically. This ensures the output remains consistent and predictable.
- **Observation on Parsing Arguments:** The program must take inputs from the command line. I observed that manually checking the exact order of command-line flags (like --file1 then --version1) is fragile. Instead, I placed all command-line arguments into an unordered_map. This allows the program to correctly identify flags no matter what order the user types them in.
- **Observation on Performance:** Creating new string copies while pulling words from a file slows down execution heavily. By passing a reference directly to the version_index's map (via get_frequencies_ref()) and utilizing std::move, the program skips creating a temporary list of words entirely. This makes the execution time faster while fulfilling the strict memory requirements. Finally, all outputs properly display the buffer size and execution time as requested.
- **Observation on Edge Cases:** I observed that a word could theoretically be larger than the buffer size and span across three or more chunks (it is not practically possible since log words are never that large). The implementation naturally handles this edge case without failing, as the left string simply continues to accumulate alphanumeric characters across multiple buffer cycles until a non-alphanumeric character is finally reached.